

Ros Day03 hw2 AI 사용 보고서

작성자: Doyeon Park

작성일: 2026.09.11

목차

1. AI 보고서 개요
2. AI 사용 부분
3. 깨달은 점

1. AI 보고서 개요

이번 과제에서는 ROS2 Topic 통신과 Qt GUI 를 이용하여 신호등 및 교차로 시뮬레이터를 제작하였다.

프로그램은 신호 상태를 관리하는 traffic_light_node, 차량의 이동을 처리하는 vehicle_node, 신호와 차량 위치를 표시하는 Qt GUI 로 구성하였다.

과제를 진행하면서 Node 별 역할, Topic 통신, YAML Parameter 및 Launch 의 개념을 이해하기 위해 AI 를 사용하였다. 또한 신호등 상태 변경, 차량 이동 및 Qt 와 ROS2 를 연결하는 코드를 작성할 때 AI 의 도움을 받았다.

AI 가 작성한 코드는 실제 패키지 구조에 맞게 직접 수정하고 실행 결과를 확인하였다.

2. AI 사용 부분

2.1 개념 이해

처음에는 신호등, 차량 및 Qt GUI 의 역할을 어떻게 분리해야 하는지 이해하기 어려웠다.

AI 에게 질문하여 traffic_light_node 는 현재 신호를 Publish 하고, vehicle_node 는 신호를 Subscribe 하여 차량 위치를 계산한다는 것을 이해하였다. Qt GUI 는 두 Node 에서 전달받은 신호와 차량 위치를 화면에 표시하는 역할만 담당한다.

또한 YAML 은 신호 유지 시간과 차량 속도를 저장하고, Launch 는 여러 Node 와 YAML 설정을 한 번에 실행하는 역할이라는 것을 알게 되었다.

2.2 코드 작성

신호등 상태 변경, 차량 이동, Topic 통신 및 Qt 화면 표시 코드를 작성할 때 AI 의 도움을 받았다.

AI 가 작성한 예제 코드를 참고하여 신호가 RED, GREEN, YELLOW 순서로 변경되도록 구현하였다. 차량은 GREEN 신호에서 이동하고 RED 신호에서 정지하도록 작성하였다. YELLOW 신호에서는 [실제로 구현한 동작]하도록 처리하였다.

신호등 점등 시간과 차량 속도는 Parameter 로 선언하고 YAML 파일에서 값을 불러오도록 작성하였다. 또한 Launch 파일을 이용하여 traffic_light_node, vehicle_node 및 Qt GUI 를 함께 실행하도록 구성하였다.

AI 의 예제에는 실제 프로그램과 다른 패키지명과 Topic 명이 포함되어 있어 내 프로젝트에 맞게 직접 수정하였다. 이후 빌드와 실행을 통해 Topic 통신과 Parameter 적용 여부를 확인하였다.

Day03 HW2의 Qt-ROS2 연결까지 완료했습니다.

- `/traffic_light_state` 구독 → 신호등 색과 텍스트 변경
- `/vehicle_position` 구독 → 위치값 표시 및 `car` 이동
- Qt 타이머로 ROS 콜백 처리
- 안전한 종료 처리 추가
- CMake에 `rcldcpp`, `std_msgs` 연결
- 최종 빌드 성공
- 실제 ROS 그래픽에서 `traffic_sim_qt` 의 두 토픽 구독 확인

수정 파일:

- `mainwindow.cpp`
- `mainwindow.h`
- `main.cpp`
- `CMakeLists.txt`

실행은 터미널 1에서:

```
</> Bash
source /opt/ros/jazzy/setup.bash
source ~/colcon_ws/install/setup.bash
ros2 launch parkdoyeon_day3_hw2 traffic_sim.launch.py
```

터미널 2에서:

```
</> Bash
source /opt/ros/jazzy/setup.bash
~/traffic_sim/build/Desktop-Debug/traffic_sim
```

기본 ROS 노드가 실행 중이라면 첫 번째 명령을 중복 실행하지 말고 Qt 앱만 실행하면 됩니다. 신호에 따라 색이 바뀌고, 위치값에 따라 자동차가 왼쪽에서 오른쪽으로 움직여야 합니다.

3. 깨달은 점

이번 과제를 통해 ROS2에서는 기능을 여러 Node로 분리하고 Topic을 이용하여 데이터를 전달한다는 것을 깨달았다.

신호등 Node는 현재 신호를 결정하고, 차량 Node는 신호에 따라 차량 위치를 계산한다. Qt GUI는 계산을 직접 수행하지 않고 ROS2 Node가 전달한 결과를 화면에 표시한다는 것을 알게 되었다.

또한 YAML Parameter를 사용하면 소스 코드를 수정하지 않고 신호 유지 시간과 차량 속도를 변경할 수 있다는 점을 알게 되었다. Launch 파일을 이용하면 여러 Node와 설정 파일을 하나의 명령으로 실행할 수 있었다.

AI는 개념을 이해하고 코드의 기본 구조를 작성하는 데 도움이 되었다. 그러나 AI가 작성한 코드가 실제 프로젝트와 완전히 같지는 않기 때문에 패키지명, Node명, Topic명 및 파일 경로를 직접 확인하고 수정해야 했다.

좋아. 그러면 Qt는 내가 만들고, 우리는 ROS2 쪽 로직부터 만들자. 과제 조건대로

`traffic_light_node`가 RED → GREEN → YELLOW → RED 상태를 관리하고, Qt는 나중에 그 Topic을 받아 표시만 하게 만들면 돼.

먼저 `traffic_light_node.cpp` 부터 가자.

현재 패키지 폴더에서:

```
</> Bash
cd ~/colcon_ws/src/parkdoyeon_day3_hw2
nano src/traffic_light_node.cpp
```

아래 코드 넣어.

```
</> C++
#include <rcldcpp/rcldcpp.hpp>
#include <std_msgs/msg/string.hpp>

#include <chrono>
#include <string>

using namespace std::chrono_literals;

class TrafficLightNode : public rcldcpp::Node
{
public:
    TrafficLightNode()
    : Node("traffic_light_node"),
      state_("RED")
    {
        // YAML에서 받을 파라미터 선언
        this->declare_parameter("red_time", 5.0);
        this->declare_parameter("green_time", 5.0);
        this->declare_parameter("yellow_time", 2.0);

        // 파라미터 값 가져오기
        red_time_ =
            this->get_parameter("red_time").as_double();

        green_time_ =
```